Available online at www.sciencedirect.com

Procedia Computer Science 00 (2026) 1–14

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

Oasis: A Mutation Testing Framework for Evaluating the Semantic Quality of Infrastructure-as-Code Test Suites

Tushar Teotia[a], Shiven Goomer[a], Yash Vardhan Rawat[a], Minakshi Tomer[a,2,*], Manoj Kumar Malik[a,1]

[a]*Department of Information Technology, Maharaja Surajmal Institute of Technology, Janakpuri, New Delhi 110058, India*

Abstract

Passing native Infrastructure-as-Code tests does not establish sensitivity to configuration faults. Oasis integrates bounded HCL mutations with selected Terraform/OpenTofu test scopes, input/default binding checks, separate assertion and runtime outcomes, and source-restoration evidence. A technically screened convenience sample yields 62 completed repository attempts and 1288 scored cases across 47 repositories: 115 assertion kills, 76 runtime-error kills and 1097 survivors. The assertion-kill fraction is 8.9%; combined rejection is 14.8%. Another 245 generated cases are excluded. Known selected-context no-ops are filtered conservatively; general semantic equivalence remains unresolved. Boolean inversion supplies 89.1% of scored cases. A separate 123-case matched study increases assertion detections from 39 to 73 through assertion and input strengthening. These exploratory results concern test sensitivity under recorded fixtures, not representative fault prevalence or deployed-cloud security.

© 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).
Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: Infrastructure-as-Code, Terraform, OpenTofu, Mutation Testing, Test Oracles, Reproducibility

1. Introduction

Infrastructure-as-Code (IaC) places infrastructure decisions under version control, but a valid configuration can still express an undesirable access, encryption or capacity setting. Research on defect-related properties in Puppet [1], security smells [2] and policy adoption in Terraform [3] motivates configuration testing without establishing how sensitive a particular project's assertions are to changes in its intended configuration.

Terraform and OpenTofu support native tests over plans and applies [4, 5]; provider mocks supply controlled values instead of remote operations [6]. A passing suite can therefore establish anything from configuration executability to a specific invariant. Mutation testing measures sensitivity by changing a program and observing its

*Corresponding author. Email: tomer.minakshi@gmail.com
[1]Assistant Professor. Email: manojmalik@msit.in. ORCID: 0000-0001-9477-5592.
[2]Assistant Professor. ORCID: 0000-0001-5536-3675.

tests [7, 8]. Applying this principle to a native IaC suite nevertheless requires distinguishing an effective configuration edit from a changed source literal that selected callers override. An already-failing baseline must also be separated from rejection caused by a mutation, and runtime rejection from an assertion that checks a requirement.

Oasis addresses these attribution problems through a bounded native-suite protocol rather than a new mutation-testing principle. Its contributions are: (i) selected-scope mutation and conservative caller/default analysis with explicit baseline, validity, outcome and restoration gates; (ii) an auditable, version-separated evaluation that reports both usable results and execution attrition; and (iii) a fixed-edit test-strengthening comparison and a historical scanner comparison that examine different oracle signals. Earlier Terraform mutation work includes the empirically motivated catalog and Terramutate prototype of Ribeiro and Neves [24]. Oasis contributes a systematic native-suite evaluation and attribution record, not the first Terraform mutation tool or a new fault taxonomy. Section 3.3 explains the supported decisions and their limits.

We address four questions: **RQ1**, which repository scopes produce usable outcomes and which are blocked? **RQ2**, how do the executed operator families and retained/added subsets behave? **RQ3**, can test strengthening change detections on identical frozen cases? **RQ4**, what costs are observed, and how do historical native outcomes relate to policy-scanner observations? The primary expansion, historical studies and matched strengthening experiment are reported separately.

2. Background and related work

IaC research relates source properties to defects [1], identifies security smells [2], and analyzes input-handling and configuration bugs across Ansible, Puppet and Chef [9]. Verdet et al. study security-policy adoption in public Terraform projects [3]. These findings motivate bounded configuration fault models; neither cross-ecosystem taxonomies nor policy categories validate the prevalence or realism of our exact mutations. GLITCH detects static IaC smells [10, 11], a complementary purpose to observing an existing suite's response to an edit.

Ribeiro and Neves [24] analyze 6,749 corrective commits from 54 public Terraform repositories and organize mutation operators into nine families using commit-message themes. Their Terramutate proof of concept injects mutations and executes associated tests in a reference project; it is therefore prior Terraform mutation testing, not merely an unimplemented taxonomy. Their systematic evaluation and assessment of whether mutant-detecting tests also detect real faults remain future work. Oasis addresses the distinct empirical question of how selected native suites respond to bounded edits across recorded repository scopes. Our evaluation likewise does not establish mutant–real-fault coupling. The studies differ in contribution and evaluation scope, not in whether mutation testing has previously been applied to Terraform.

Table 1. Different testing targets in related work. This is a capability/estimand comparison, not a controlled speed or accuracy ranking.

Approach	Object and oracle	Relationship to Oasis
Terramutate [24]	Terraform operator catalog motivated by corrective-commit themes; reference-project mutation/test proof of concept	Prior Terraform mutation testing. Oasis adds a separately documented multi-repository native-suite evaluation; neither study establishes mutant–real-fault coupling.
ProTI [12]	Pulumi TypeScript programs; generated configurations, mocks and pluggable property oracles	Generates test configurations rather than evaluating an existing native HCL suite against configuration-changing edits.
EMIaC [13]	IaC engines; consistency under semantics-preserving program transformations	Targets engine inconsistencies; Oasis instead observes suite rejection of intended configuration perturbations.
Oasis	Native Terraform/OpenTofu suites; existing assertions and runtime outcomes	Combines literal edits, selected-scope and effective-input gates, outcome separation and recorded restoration.

ProTI generates and checks configurations for Pulumi TypeScript programs using mocks and pluggable oracles [12]. EMIaC uses semantics-preserving transformations to test IaC engines [13]. Oasis instead changes configuration values and observes an existing native suite. The subjects, transformations and oracles therefore differ

(Table 1); this distinction supports a bounded integration contribution, not a claim that mutation testing or IaC testing is wholly unexplored.

Mutation surveys discuss operator selection, execution cost and semantic equivalence [7, 8, 14]. Application-code research investigates coupling between mutant and real-fault detection [15], limitations of coverage as an effectiveness proxy [16], industrial testing practice [17, 18], and practitioner adoption [19]. Such work motivates the technique but supplies no universal adequate-score threshold for this corpus.

Table 2. Different experimental purposes preclude a common mutation-score ranking.

Study	Evidence focus	Comparison limit
Application mutation [15]	Mutant–real-fault relationship	No comparable real Terraform fault oracle here.
Industrial mutation [17, 18]	Testing practice and selective execution at scale	Different operators, sampling and filtering.
Oasis	1288 primary cases; separate 123 matched cases	Native mock/fixture sensitivity; clustered observations.

3. Oasis design

3.1. Execution pipeline and outcome gates

Oasis is implemented in Python. It identifies supported literal spans in tracked local HCL, records the file, source offset, original and replacement values, and source hashes, and applies one edit at a time without reserializing surrounding configuration. A scope comprises a repository, module directory and selected test directory. Downloaded registry modules can execute as dependencies but lie outside the local-source mutation pool. The same edit exercised through two scopes is two mutant cases, not two independent real defects.

The pipeline initializes dependencies, runs unmutated native tests and standalone validation, checks the candidate's effective input context, validates each mutation, invokes native tests, classifies the outcome and restores source bytes (Fig. 1). A baseline failure is never a mutant kill. Timeouts, invalid candidates, unresolved contexts and known no-ops remain explicit exclusions. Each outcome is attached to the source hash, operator, selected scope and command receipt rather than only a pooled score. This preserves the distinction between an attempted edit, an eligible test case and an observed rejection.

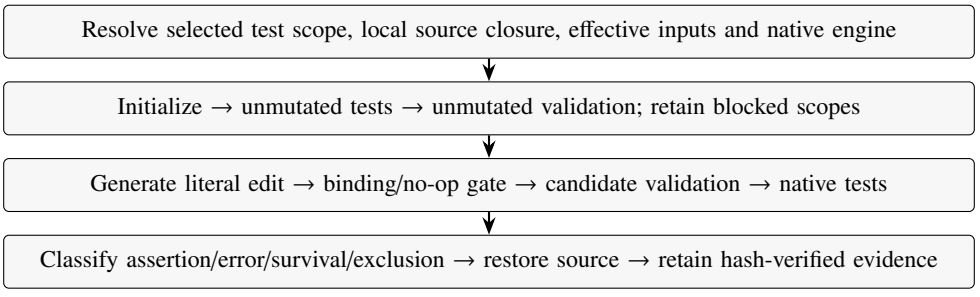


Fig. 1. Native mutation pipeline. Static potential reachability is not dynamic coverage.

3.2. Frozen catalog and fault-model boundaries

The primary study freezes framework `0.3.0-research.1`, catalog 3, 16 identifiers and 22 source-file hashes before native expansion. Twelve retained identifiers and four added identifiers are described in Table 3. Identical offset/replacement/deletion edits are counted once per scope, with the earlier specialized operator retaining attribution. The added generic Boolean operator targets supported resource/local literals, module input arguments (M), and scalar variable defaults (V). This extends bounded module-input coverage, not arbitrary module-interface semantics.

Table 3. Frozen catalog 3. Transformations are bounded fault hypotheses, not a validated distribution of real bugs. R: resource body, also accepted module bodies for unscoped operators; V: default; L: local; M: explicit module-argument label for the generic Boolean operator.

Identifier(s)	Transformation	Surface
MUT-SEC-001	S3 private ACL to public-read	R
MUT-SEC-002	Supported storage encryption true to false	R
MUT-SEC-003	Restrictive literal CIDR to 0.0.0.0/0	R
MUT-SEC-004a–d	Disable one of four S3 public-access-block flags	R
MUT-RES-001	Supported instance-size literal to a larger class	R,V
MUT-RES-002	Supported count/desired/minimum of at least two to one	R,V
MUT-RES-003	Positive replica literal to zero	R,V
MUT-TAG-001	Environment production to development	R,V,L
MUT-TAG-002	Delete a literal team/owner/cost_center tag	R,L
MUT-CFG-001 (added)	Boolean literal inversion, including supported caller/default sites	R,V,L,M
MUT-RES-004 (added)	Curated positive capacity/count literal to zero	R,V
MUT-SEC-005 (added)	Scoped EC2 IMDS tokens required to optional	R,V
MUT-SEC-006 (added)	Scoped EFS/Redshift encryption true to false	R

Table 4 separates the documented meaning of an edit, its motivating fault category and the claim that can be made from it. Access/encryption transformations relate conceptually to security-policy categories [3]; input perturbations relate to broader IaC configuration-fault research [9]. These are design motivations, not a validated distribution of real Terraform defects. Generic Boolean inversion is not automatically a security fault, and metadata edits concern project-specific contracts. Literal matching, resource restrictions, unsupported expression/object forms, remote dependencies and shared module structure constrain the model. Native OpenTofu execution is supported, but no OpenTofu-specific semantic rewrite family is evaluated.

Table 4. Fault hypotheses and the evidence that motivates them. A documented policy or language rule establishes the meaning of a change, not the prevalence of that change among real defects.

Operator group	Motivating contract or mechanism	Inference boundary
Access and encryption	Terraform security-policy categories motivate restricting access and retaining encryption controls [3].	A changed control can be redundant or irrelevant to the selected configuration; a scanner finding is not deployed-vulnerability ground truth.
Boolean inputs/defaults	IaC configuration/input faults motivate perturbing supplied values [9]. The native input rules determine whether a default is effective [21].	Inversion is not inherently harmful. The project assertion, not the Boolean type, defines the observed contract.
Capacity and replicas	Provisioning expectations motivate minimum/count and replica perturbations; configuration-fault research supplies category-level motivation [9].	Zero or a larger instance can be intentional; literal matching alone does not establish a real fault.
IMDS token policy	Required tokens restrict metadata access to IMDSv2, whereas optional tokens permit IMDSv1 or IMDSv2 [22].	This documents the semantic relaxation for a scoped operator, not representative frequency or end-to-end exploitability.
Metadata tags	A project can require ownership, cost-centre or environment metadata.	No universal security or defect claim is assigned to a missing or changed tag.

3.2.1. Correspondence with a Terraform-specific fault catalog

Table 5 maps every frozen identifier to the variants of Ribeiro and Neves [24]. Oasis mainly instantiates portions of their parameter, variable-default and resource-configuration families. This is a source-level comparison of stated transformations, not an experiment on their mined defects. Boolean inversion at resource/module arguments corresponds directly to `PAR_BOOL`, while changes to variable defaults correspond to `VDR_DEF`. Resource-tag deletion and literal CIDR widening have explicit counterparts. In contrast, our capacity-to-zero/one mutations are not their

general ±1 numeric edit, and a curated instance-class replacement is not a typographic string error. Their generic RCR_ATTR category supplies only broad motivation for edits such as ACL and IMDS-policy changes, not a validated match to a historical defect.

The published catalog traces families to commit-message clusters; its largest cluster contains 5,052 general-maintenance messages, and its limitations acknowledge that message semantics and language knowledge do not replace patch-level validation. Accordingly, the mapping supports a documented fault-model rationale without establishing operator frequencies, representative coverage or predictive accuracy for real bugs. Oasis does not implement their version-constraint/literal, scanner-suppression, IAM-policy, provider-declaration or module-declaration families, nor their variable-type/sensitivity and whole-resource deletion variants. Its tested contribution is deliberately narrower: sensitivity of existing suites to selected configuration edits.

Table 5. Analytical correspondence between all 16 frozen Oasis identifiers and the catalog of Ribeiro and Neves [24]. Direct means a shared stated edit at a compatible locus, not validated defect coverage; partial means related but nonidentical edits; generic means only their broad attribute-change family applies.

Oasis identifier	Prior variant(s)	Correspondence and boundary
MUT-SEC-001	RCR_ATTR	Generic: their broad attribute change does not specify the private-to-public ACL edit.
MUT-SEC-002	PAR_BOOL; RCR_ATTR	Direct Boolean inversion; Oasis restricts eligible storage types/attributes.
MUT-SEC-003	RCR_SG_OPEN	Direct for literal CIDR widening; Oasis does not implement their all-ports alternative.
MUT-SEC-004a	PAR_BOOL	Direct Boolean inversion of block_public_acls.
MUT-SEC-004b	PAR_BOOL	Direct Boolean inversion of block_public_policy.
MUT-SEC-004c	PAR_BOOL	Direct Boolean inversion of ignore_public_acls.
MUT-SEC-004d	PAR_BOOL	Direct Boolean inversion of restrict_public_buckets.
MUT-RES-001	RCR_ATTR; VDR_DEF	Generic on resource attributes; direct default alteration on V. A curated larger class is not a string typo.
MUT-RES-002	PAR_NUM; VDR_DEF	Partial: count/capacity to one, not general ±1; direct default alteration on V.
MUT-RES-003	PAR_NUM; VDR_DEF	Partial: replicas to zero, not general −1; direct default alteration on V.
MUT-TAG-001	RCR_ATTR; VDR_DEF	Generic on resource attributes; direct default alteration on V. Module/local relabeling has only conceptual overlap.
MUT-TAG-002	RCR_TAG_DEL	Direct on resource tags required by the project; module/local sites extend the locus. No generated targets here.
MUT-CFG-001	PAR_BOOL; VDR_DEF	Direct on resource/module arguments and Boolean defaults; local literals extend the locus.
MUT-RES-004	PAR_NUM; VDR_DEF	Partial: capacity to zero, not general ±1; direct default alteration on V.
MUT-SEC-005	RCR_ATTR; VDR_DEF	Generic on resource attributes; direct default alteration on V. No dedicated prior IMDS variant.
MUT-SEC-006	PAR_BOOL; RCR_ATTR	Direct Boolean inversion; Oasis restricts the edit to EFS/Redshift encryption.

3.3. Equivalence and selected-context binding

Survival does not establish semantic equivalence. The frozen catalog 3 resolver conservatively analyses caller/default binding within the selected local test closure. Its purpose is to avoid attributing a source-level default edit to a test context that supplies an independent value, while retaining supported routes where the default can matter. **Supported decision procedure.** The frozen resolver classifies variable-default edits against the selected test context; it does not evaluate arbitrary HCL or solve semantic equivalence.

1. Resolve the selected native test files and repository-local caller closure, respecting engine-specific file precedence. Discovery fallback is not used as proof of omitted inputs or nullability.
2. For a scalar-default edit, record each supported root-run or nested-caller binding as active, shadowed or unresolved (Table 6). File-level and run-level test inputs are combined with run values taking precedence. Language rules explain default/null semantics [21]; the analyzer can be deliberately more conservative.

3. If analysis fails, return unresolved. Otherwise, any active record makes the candidate eligible; only an all-shadowed record set proves a selected-context no-op. With neither condition, retain an unresolved exclusion. Thus an active route can coexist with other unresolved routes.
4. After a passing unmutated baseline, the runner maps shadowed to `no_op`, unresolved to `scope_unresolved`, and active to normal mutant validation/testing. Keep assertion failure, runtime error, survival and timeout separate, and retain restoration evidence.

Table 6. Supported frozen binding decisions for a scalar default. "Active" means the default can matter, not that the run or resource executes. Root-run checks are stricter than nested-caller checks.

Input evidence	Nested caller	Selected root run
Argument absent	Active	Active only when external root-input influence is absent; otherwise unresolved.
Independent non-null literal	Shadowed	Shadowed only when all effective supplied input values are recognized literals.
Explicit null; non-nullable default	Active	Active.
Explicit null; nullable input	Shadowed	Unresolved.
Expression supplied	Shadowed for nullable child; otherwise unresolved	Unresolved.
Unparsed/unsupported binding, including JSON configurations	Unresolved	Unresolved.

For illustration, an independent literal supplied to a child module can shadow a changed default, whereas omitting it can expose the edit. At a root test run, other expression-valued inputs can prevent proof of shadowing even when the target input is literal. Matching environment/variable-file/CLI input influence is likewise not guessed. These are bounded analysis decisions, not additional measured results. Survivors are not removed because a scanner is silent, direct static references are absent, or mocks obscure a value. Unknown equivalence remains a denominator limitation; historical `no_coverage` labels describe static patterns, not measured dynamic resource coverage.

4. Empirical method

4.1. Selection, cohorts and deadline cutoff

The primary plan contains 95 distinct canonical upstream identities: 37 earlier additions, 35 technically applicable reserves from 40 completed static screens, and 23 original sources retained in their historically adapted form. Discovery and manual inspection favor native tests, mock-compatible execution and local configurations with potential frozen-operator sites. Static screening stops at a first unshadowed potential site and records a lower bound; it neither predicts the final mutant count nor substitutes for native baseline gates. Admission does not use a kill-rate threshold; earlier campaign outcomes are nevertheless visible. This convenience sample includes reusable modules, platforms, examples and deliberately vulnerable fixtures; it is not representative of Terraform users.

A time-based cutoff closed admission before the final LangChain GCP and n8n outcomes were known. Only those already active repositories were permitted to finish; pending preparation/environment retries and unstarted work were not continued. Earlier outcomes and runtimes were visible, and scheduling favoured smaller observed workloads. This is disclosed selection and attrition, not random sampling, a blind holdout or outcome-independent preregistration. The reporting snapshot of 9 November 2026 contains 62 completed archives, 3 failed preparations with retries cancelled, one in-progress repository excluded from results, and 29 unstarted repositories. The original plan and all dispositions are retained.

4.2. Source policy, engines and execution

The catalog 3 expansion introduced no new adaptations to upstream assertions, mocks or test values. The original 23-source admission cohort, however, used historically adapted sources and is not a pristine-upstream cohort. Earlier preparation included synthetic inputs/mocks, dependency/source repairs, some assertion-expression repairs and

removal of assertions whose complete condition was a constant true or false. A retained recipe is not proof that every conditional repair was applied. Table 7 separates the recorded phase policy from historical change accounting; the supplement distinguishes manifest-verified final bytes, recipe-only evidence and missing applied ledgers. Complete per-category change totals are unknown, not zero.

Table 7. Source-policy accounting for the 62 completed catalog 3 attempts. All archive receipts record no assertion changes in this phase; that field does not certify pristine upstream tests or unchanged historical assertions.

Admission cohort	Completed	Interpretable source policy
Historically adapted original sources	15	Earlier synthetic inputs/mocks, compatibility repairs and some assertion changes were retained. No further catalog 3 adaptations were recorded.
First 37 additions	20	No new assertion, mock or test-value adaptations were introduced in the recorded catalog 3 phase.
Applicability-screened reserves	27	Same frozen phase policy; baseline success and selected-scope completeness remain separate outcomes.

Primary native execution used Terraform 1.16.7 or OpenTofu 1.12.3, two execution lanes, serialized initialization, a 900-second initialization limit and 600-second test/validation limits. Every eligible mutant in a baseline-cleared selected scope was attempted without a mutant-count cap. Source restoration was checked after scopes. Immutable, SHA-verified provider snapshots isolated running commands from concurrent package-cache writes. Native evaluation was offline except for local provider IPC; downloads occurred during initialization. The recorded protocol used OS write/private-file restrictions, no cloud credentials and no live cloud provisioning. Setup failures, native test failures, unresolved contexts and timeouts were retained rather than repaired by weakening assertions.

Interrupted attempts were retained outside active counts. Scope reuse required matching selected commands, source revisions/file hashes, frozen code hashes and manifest/artifact verification. An auxiliary GCP baseline timing probe ran no mutants and was excluded from primary cases, scope completions and command-cost summaries. Operational recovery details are retained in the evidence records rather than treated as additional experimental observations.

4.3. Metrics and denominators

Let A denote assertion kills, E runtime-error kills and S survivors among valid, baseline-cleared scored cases:

$$N = A + E + S, \qquad D_A = A/N, \qquad MS = (A + E)/N. \tag{1}$$

D_A measures observed assertion sensitivity; MS additionally includes runtime rejection. Timeout is not credited as either kind of kill. Invalid, no-op, unresolved, timeout and execution-error mutant records are separately excluded. Baseline failures and target-free module rows are scope dispositions, not generated mutant cases. Repositories with $N = 0$ receive n.a., not a zero score. Pooled case fractions, repository-level summaries and all-selected-baseline coverage answer different questions. A finished scored repository can still contain blocked scopes.

The retained 12 and added 4 subsets use the same catalog 3 engine, inputs, resolver and dependency context. This is a within-version operator partition, not a controlled catalog 2 rerun. Historical cases, the matched strengthening pool and the primary expansion never share one mutation denominator. We report canonical identity unions only as provenance, not as a combined scored sample.

4.4. Separate historical and matched studies

The original catalog 2 execution-adapted study covered 23 repositories and 336 scope dispositions: 306 passed baselines, 27 failed and 3 lacked dependencies. It generated 299 cases, with 276 scored (47 assertion kills, 2 runtime kills and 227 survivors), 21 unresolved, 1 invalid and 1 proven context no-op. A prior 37-repository catalog 2 expansion contributed 283 scored cases in 13 repositories; exact-environment recovery added Worldcoin's 3 scored survivors. These historical sets account for 29 scored identities, with their original manifests, engine versions and preparation policies; their mutation percentages are not pooled with catalog 3.

A separate experiment froze 123 matched cases across 40 modules in 8 repositories. Both arms used exactly the same edits. Source-informed strengthening added 63 assertions to existing runs and 37 positive-control runs while preserving original runs, mocks, overrides and source. Because added runs expanded the exercised input contexts, the treatment is test strengthening, not an isolated assertion-only intervention. All 40 strengthened baseline/control gates passed. The development subset contained 95 cases and the transfer subset 28 cases from 2 repositories. Proposals were sealed before transfer execution, but prior pilot exposure and investigator-authored treatment preclude a pristine blind-holdout claim. All 80 arm jobs finished, yielding 246 executions and reproducing the original labels. Preservation checks for this revision verified bytes and paired outcomes, not fresh execution of those jobs.

4.5. Policy comparison and timing protocol

The policy comparison uses the historical original 23 catalog 2 snapshots, not the new primary cohort. tfsec 1.28.14 [20] scans copied, hash-verified sources without module downloads. Unmutated/mutant module comparisons associate newly reported rule identifiers with findings overlapping the mutation line. Existing baseline findings receive no detection credit. Location association is incomplete semantic evidence and is not vulnerability ground truth; scanner results never become assertion kills.

Primary command receipts separate initialization, baseline tests and validation, and the validation and test command for each mutant. We report elapsed seconds, medians and empirical nearest-rank 95th percentiles from recorded commands, including failures and timeouts. Summed commands overlap across lanes and omit some preparation, queue wait, snapshot and framework overhead; they are not wall time. Historical command costs retain their own cohort/version. Repeated-run variance, deployment cost savings and engine performance differences are not measured.

5. Results

5.1. RQ1: completion, coverage and exclusions

Table 8 gives the primary disposition ledger. 192 archived scope attempts comprise 130 passing and 62 blocked/unavailable baselines. 47 completed repositories contribute scored mutants: 39 pass every selected baseline; 8 contribute scored cases from some scopes while others are blocked. Another 15 completed repository attempts have no scored mutants. A repository count alone therefore overstates complete execution coverage if these categories are collapsed.

Table 8. Disposition of the 95-repository plan at the reporting snapshot. Only completed archives contribute mutant outcomes.

Disposition	Count
Completed repositories with scored mutants: all selected baselines pass	39
Completed repositories with scored mutants: some baselines blocked	8
Completed repositories without scored mutants	15
Preparation failures; retries cancelled	3
In progress; excluded from mutation totals	1
Not started before the deadline cutoff	29
Total planned repository identities	95
Completed scope attempts: passing / blocked baseline	130 / 62
Generated cases: scored / excluded	1,288 / 245

Table 9. Catalog 3 outcomes by admission cohort. The sources and sampling policies differ; these are not matched groups.

Admission cohort	Completed	Scored repos	N	D_A
Historically adapted original sources	15	11	507	7.9%
First 37 additions	20	14	379	10.6%
Applicability-screened reserves	27	22	402	8.7%

There are 1533 generated mutant-case records: 1288 scored and 245 excluded. Exclusions comprise 193 known context no-ops, 39 unresolved contexts, 13 timeouts and 0 other invalid/execution-error records. Baseline-blocked scopes can have unknown potential mutant counts; the excluded-case total is not a census of every counterfactual mutation in unexecuted scopes. The repository/scope/case CSVs retain every admitted identity and unavailable outcome.

LangChain illustrates the distinction between feasibility and assertion sensitivity. Its AWS scope completed 133 eligible native attempts, including 13 timeouts, after 39 shadowed candidates were excluded. Azure initialization encountered HTTP 504 while fetching provider authentication checksums and did not reach baseline testing; its retry was cancelled under the cutoff. Completed AWS/GCP records supplied 233 scored cases. The unfinished n8n repository contributed no primary outcomes at this snapshot. These observations describe the attempted environment, not Azure assertion weakness or a completed three-cloud success. Other blockers included missing source, provider availability/startup errors, offline external-program requirements and native source/test disagreements.

5.2. RQ2: operator and family outcomes

Primary outcomes are 115 assertion kills, 76 runtime-error kills and 1097 survivors, giving D_A =8.9% and MS =14.8%. The unweighted mean repository combined fraction is 33.5% and its median is 18.2% over repositories with scored cases. These descriptive summaries do not turn small clustered samples into independent population observations.

Table 10. Operator-family outcomes in catalog 3. Families partition scored cases, not independent real defects.

Family	N	A	E	S	D_A
Configuration	1147	102	51	994	8.9%
Security	67	9	0	58	13.4%
Resource	72	3	24	45	4.2%
Compliance	2	1	1	0	50.0%

Table 11. Within-catalog 3 operator partitions use the same execution machinery. This is not a catalog 2 versus catalog 3 performance comparison.

Subset	N	A	E	S	D_A	MS
Retained 12 identifiers	77	9	2	66	11.7%	14.3%
Added 4 identifiers	1211	106	74	1031	8.8%	14.9%

Table 12. All 16 identifiers, including the zero-target identifier. $D_A = A/N$ is the assertion-kill fraction; $MS = (A + E)/N$ additionally credits runtime rejection. Exclusions do not enter either denominator; n.a. denotes no scored cases.

Identifier	Gen.	N	A	E	S	Excl.	D_A	MS
MUT-CFG-001	1363	1147	102	51	994	216	8.9%	13.3%
MUT-RES-001	19	9	0	1	8	10	0.0%	11.1%
MUT-RES-002	8	4	0	0	4	4	0.0%	0.0%
MUT-RES-003	3	3	0	0	3	0	0.0%	0.0%
MUT-RES-004	67	56	3	23	30	11	5.4%	46.4%
MUT-SEC-001	1	1	0	0	1	0	0.0%	0.0%
MUT-SEC-002	4	4	1	0	3	0	25.0%	25.0%
MUT-SEC-003	2	2	0	0	2	0	0.0%	0.0%
MUT-SEC-004a	14	13	3	0	10	1	23.1%	23.1%
MUT-SEC-004b	14	13	2	0	11	1	15.4%	15.4%
MUT-SEC-004c	14	13	1	0	12	1	7.7%	7.7%
MUT-SEC-004d	14	13	1	0	12	1	7.7%	7.7%
MUT-SEC-005	7	7	1	0	6	0	14.3%	14.3%
MUT-SEC-006	1	1	0	0	1	0	0.0%	0.0%
MUT-TAG-001	2	2	1	1	0	0	50.0%	100.0%
MUT-TAG-002	0	0	0	0	0	0	n.a.	n.a.

The generic Boolean operator contributes 1147 cases (89.1% of scored cases), so larger sample coverage mainly reflects configuration-literal sensitivity. The retained 12 subset contains 77 scored cases; the added 4 subset contains 1211. Their different fractions reflect different targeted faults and sites, not a causal effect of operator design or a version-to-version change in suite quality. Scored cases comprise 1049 distinct repository-qualified edit identities; the additional cases re-exercise edits through different selected scopes. Neither number is a count of independent real defects. Security/resource families and zero-target identifiers remain separately visible.

Table 13. Recorded catalog 3 cases. C2 names the variable whose default changed; C3 identifies the resource field, not an inferred timeout cause. Source locations and case/command identities are retained in the evidence package.

ID	Repository	Recorded edit	Native outcome
C1	ambicuity/terraform-aws-eks-ray	`aws_kms_key.eks`: `enable_key_rotation`, true → false	Survived
C2	equinor/terraform-azurerm-key-vault	Default of `purge_protection_enabled`, true → false	Assertion kill
C3	langchain-ai/terraform	`random_password.in_cluster_postgres`: `special`, false → true	Timeout; excluded

Source-to-oracle example (C2). The Equinor module's `purge_protection_enabled` variable has default true in `variables.tf:32`; `main.tf` passes it to `azurerm_key_vault.this`. The existing `soft_delete_defaults` run uses a mocked Azure provider and a plan command. Its inputs come from `setup_tests`, but omit the purge-protection input. The retained test at line 26 checks:

```
azurerm_key_vault.this.purge_protection_enabled == true
```

Inverting the default to false remained valid. The native diagnostic identifies that exact assertion, reports the planned value as false, and ends with 24 passed and 1 failed; the unmutated suite reports 25 passed. The explicit-true and explicit-false purge-protection runs still passed. Binding receipts mark the default-checking route active but the explicit-input runs unresolved because other setup-derived inputs are expressions; their successful native execution is not misreported as static proof of shadowing. The verified source/test/log hashes and command context are supplied in `polish-evidence/worked-example-evidence.json`.

Purge protection prevents purging a deleted vault/object until its retention period passes [23]. That documented meaning motivates the field's relevance; the measured finding is narrower: an existing mocked-plan assertion detects

a default-contract violation. It is not a deployed incident or a real-fault validation sample. C1 records acceptance of a KMS rotation edit without establishing why it survived. C3 records a password-field edit followed by a timeout; it establishes neither a faulty assertion nor the cause of the timeout and is excluded from scoring.

A survivor shows only acceptance under recorded inputs and mocks. Missing or indirect oracles, inactive conditions, mock behavior, irrelevant contracts and semantic equivalence are possible explanations. If e scored survivors were later proved equivalent, the assertion fraction would be $A/(N - e)$; no value of e is inferred from survival or scanner silence. Neither unresolved outcomes nor unknown mutants in baseline-blocked scopes are imputed as detected or undetected.

5.3. RQ3: frozen test strengthening

Table 14. Separate fixed 123-case comparison; every pair is scored.

Subset	Cases	Original A	Strengthened A	New detections
Development	95	34	56	22
Transfer (two repositories)	28	5	17	12
Total	123	39	73	34

Assertion detections rise from 39/123 (31.7%) to 73/123 (59.3%), a 27.6 percentage-point gain, with 34 new and no lost detections. Two runtime-error kills remain in each arm; survivors fall from 82 to 48. Gains concentrate in `platform-tools` (20), `serverless-architecture-patterns` (12) and `platform-design` (2). Thirty-two gains concern S3 public-access-block edits; two concern replica-to-zero edits. All 12 transfer gains occur in `serverless-architecture-patterns`; the second transfer repository contributes one already-killed case. The result demonstrates changed sensitivity to fixed edits under the combined assertion/input intervention, not dynamic coverage, general transfer to real bugs or an independence-based significance result.

5.4. RQ4: costs and historical scanner observations

Table 15. Observed native command durations (seconds), including failures and timeouts. Interrupted attempts and the auxiliary baseline probe are excluded; sums are not wall time.

Phase	Commands	Sum (s)	Median (s)	P95 (s)
Initialization	191	1,641.1	4.57	24.97
Unmutated test	165	3,102.2	13.21	35.11
Unmutated validation	131	571.1	3.06	14.17
Mutant validation	1301	8,426.9	5.57	14.17
Mutant test	1301	94,002.5	11.59	386.88

Table 15 includes native commands from archived primary attempts and excludes interrupted history and the separate probe. Cost skew and failed setup are part of feasibility. A warm-cache fast baseline is not a full mutation-run forecast: the number of effective sites, failing paths and timeouts also matters. Exhaustive execution on important modules can be expensive; this study does not establish a mandatory routine-adoption policy or validate a cost-reduction scheme.

The following scanner observations use only the historical 276-case original 23 study. They are retained to answer the review request for a named scanner on corresponding snapshots, not extrapolated to catalog 3. The repository-root scanner invocation completes with structured output for 20 of 23 snapshots; three fail to produce usable JSON. Root-level finding counts are not a measure of complete resource coverage. At the case level, 231 unmutated/mutant comparisons are usable and 45 are unavailable (16 GenAI and 29 Psoxy cases), principally because this scanner rejects newer HCL blocks such as `check` and `removed`. Native test results for those cases remain valid; scanner failures are not treated as policy nondetections. Table 16 retains all cases.

Table 16. Joint native-test/scanner outcomes on expanded cases. "New rule" means a newly reported rule identifier overlapping the mutation line; it is not vulnerability ground truth.

Scanner comparison	Assertion kill	Runtime kill	Survivor
Usable: new site rule	38	0	88
Usable: no new site rule	9	2	94
Unavailable comparison	0	0	45
Total	47	2	227

Among the 231 usable comparisons, 126 have a new site-associated policy rule, including 88 cases accepted by native tests. Nine assertion kills have no new site-associated rule. This illustrates differing observations under the recorded conditions, not scanner superiority or exhaustive complementarity: tfsec does not consume the native test runs and synthetic test inputs, default ignore behavior is retained, downloaded modules are excluded, and line association can miss relevant findings. The scanner's parser failures further limit coverage. The individual repository findings, failed diagnostics and case associations are available in the supplement. For a concrete historical contrast, changing `block_public_acls` from true to false at `bootstrap/main.tf:247` in `aws-platform-starter` survived native tests while newly triggering `aws-s3-block-public-acls`. The same field edit at line 82 of `terraform/modules/aws-ml-artifact-store/main.tf` in `platform-design` was an assertion kill without a new site-associated rule. These catalog 2 records illustrate differing observations, not deployed-vulnerability ground truth; exact mutant identities and receipts are retained in the historical supplement.

Relating survivors to prior misconfiguration evidence.. Terraform security-policy adoption [3] and the corrective-commit themes of Ribeiro and Neves [24] provide context for access, encryption and configuration edits, but study different objects from native-test survivors. In our historical comparison, 88 survivors acquire a site-associated policy finding: test acceptance and a policy warning can coexist. Conversely, nine assertion kills have no new site-associated finding, showing that scanner silence does not establish acceptance of a project's tested contract. These observations concern the same recorded snapshots, not an inferred deployment vulnerability. The 45 unavailable comparisons are missing evidence, not policy-clean configurations. We do not transfer published security-policy prevalence to our Boolean-dominated corpus or attribute an unavailable scanner run to a specific cross-file-analysis defect without its own diagnostic evidence.

6. Discussion and threats to validity

Construct validity. Oasis measures sensitivity to a bounded literal fault model. General Boolean edits, source restrictions and operator overlap shape the denominator; a family name is not proof of a real vulnerability. The literature mapping in Table 5 is an analytical comparison, not patch-level verification or measured coupling to real faults. Runtime rejection differs from an assertion oracle. Selected-context binding removes some no-ops but does not solve semantic equivalence or measure dynamic resource/attribute coverage. Policy findings, survivors and deployed-cloud vulnerabilities are distinct observations.

Internal validity. Fixtures, mock outputs, historical source and assertion repairs, aliases, dependency re-resolution and timeouts affect outcomes. Some historical assertions were repaired or removed; missing applied ledgers prevent exhaustive category counts or a claim that every repair preserved the upstream oracle. No further assertion/mocking/value adaptations were recorded in catalog 3, but this does not make the inherited sources pristine. A passing mock baseline cannot certify a live deployment. Native early failure can skip later runs. Hash checks support restoration and evidence identity, not freedom from provider/engine defects.

Selection and statistical validity. Public mock-compatible projects, shared providers/modules, vulnerable fixtures, short-workload ordering and the visible-results deadline cutoff limit generalization. Unrun and preparation-failed repositories remain in the selection ledger. Cases repeat edits across scopes and cluster by repository/family; pooled fractions and repository summaries are descriptive. A narrow binomial interval would not address nonrandom selection. The matched experiment has development exposure, a small non-blind transfer set and investigator-authored treatment; it cannot demonstrate universal assertion improvement.

Comparison and performance validity. Catalog 3 adds sites/operators and changes conservative binding relative to historical runs. The retained 12 view is a within-catalog 3 subset, not a fair historical-version benchmark. Static scanner location matching can miss indirect findings or associate unrelated checks; parser failures and unequal native/scanner input contexts prevent precision/recall claims. Timings are single observations with warm caches, concurrency and excluded overhead. The auxiliary third-lane baseline is not mixed into primary costs. No estimate is presented as repeatable cross-engine performance.

7. Reproducibility and future work

The source repository is https://github.com/DegenerateUSER/Oasis. Primary execution froze tool 0.3.0-research.1/catalog 3 with 22 source hashes supplied in the evidence package. Retained records include exact source revisions, operator/site identities, selected commands, provider locks, CSV exports, diagnostics, native timings, restoration checks and SHA-256 archive/member inventories. The primary archive audit verified 8186 source files and 30828 members. The 95-entry plan and all deadline dispositions are supplied. Identity deduplication gives 69 scored identities across historical/current versions, but this union is not a shared mutation denominator.

Historical execution revision `d535bee1ef9bb1f62c9c258dd67ead715ba826b3` and matched-harness revision `7124220311bf35dcf77b6a3438fdb8cb029b59a2` identify local research revisions, not a claimed new public release or DOI. The frozen 123-case CSV hash and original receipts are retained. Framework validation records report 242 passed checks and 4 skipped opt-in provider checks, plus native module/default/capacity canaries excluded from research; these records were not recreated by compiling this paper. The editable paper/evidence package contains processed records, not a self-contained native reproduction environment. Complete native archives remain on local/external storage and require a separate reviewer-access arrangement. Credentials and provider binaries are not included.

Future work should evaluate engine-specific constructs, broader interfaces, expression/object mutations, dynamic effective coverage, manually assessed survivor causes, independent cohorts and real-fault coupling. A coupling study would require traceable faulty/fixed revisions and comparable tests, explicit candidate exclusions and controls for test-suite size and exercised behavior; a taxonomy correspondence alone does not supply that evidence. Additional scanners and repeated cost measurements likewise require separate controlled experiments. These extensions are outside the frozen study reported here.

8. Conclusion

The primary frozen-catalog 3 study archives 62 repository attempts; 47 yield 1288 scored cases with 115 assertion kills, 76 runtime-error kills and 1097 survivors. Full selected-baseline coverage and partial coverage remain separate, as do 245 excluded mutant records, blocked scopes and unrun repositories. Generic Boolean mutations dominate the pool, constraining security-specific conclusions. A separate matched 123-case study increases assertion detections from 39 to 73 without lost detections. These results support mutation feedback as a diagnostic aid for selected native IaC oracles, with explicit attention to fixtures, fault models, attrition and cost. They establish neither representative security prevalence nor a universal adequacy threshold.

Declaration of AI assistance

OpenAI Codex and Hermes Agent (Nous Research) assisted implementation, analysis support, manuscript drafting and LaTeX preparation. Claude (Anthropic) assisted literature discovery and organization. AI-generated outputs do not independently validate citations or experimental results. Responsibility for the research record, interpretation and final text rests with the authors.

Acknowledgements

The authors acknowledge the Department of Information Technology, Maharaja Surajmal Institute of Technology, Janakpuri, New Delhi.

References

[1] A. Rahman and L. Williams, Source code properties of defective infrastructure as code scripts, Information and Software Technology 112 (2019) 148–163. https://doi.org/10.1016/j.infsof.2019.04.013.

[2] A. Rahman, C. Parnin, and L. Williams, The seven sins: Security smells in infrastructure as code scripts, in: ICSE, 2019, pp. 164–175. https://doi.org/10.1109/ICSE.2019.00033.

[3] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, Assessing the adoption of security policies by developers in terraform across different cloud providers, Empirical Software Engineering 30 (2025), article 74. https://doi.org/10.1007/s10664-024-10610-0.

[4] HashiCorp, Tests, Terraform language documentation. https://developer.hashicorp.com/terraform/language/tests. Accessed 7 November 2026.

[5] OpenTofu, Command: test, OpenTofu documentation. https://opentofu.org/docs/cli/commands/test/. Accessed 7 November 2026.

[6] HashiCorp, Tests: provider mocking, Terraform language documentation. https://developer.hashicorp.com/terraform/language/tests/mocking. Accessed 7 November 2026.

[7] Y. Jia and M. Harman, An analysis and survey of the development of mutation testing, IEEE Transactions on Software Engineering 37 (5) (2011) 649–678. https://doi.org/10.1109/TSE.2010.62.

[8] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, Mutation testing advances: An analysis and survey, Advances in Computers 112 (2019) 275–378. https://doi.org/10.1016/bs.adcom.2018.03.015.

[9] G.-P. Drosos, T. Sotiropoulos, G. Alexopoulos, D. Mitropoulos, and Z. Su, When your infrastructure is a buggy program: Understanding faults in infrastructure as code ecosystems, Proceedings of the ACM on Programming Languages 8 (OOPSLA2) (2024), article 359. https://doi.org/10.1145/3689799.

[10] N. Saavedra and J. F. Ferreira, GLITCH: Automated polyglot security smell detection in infrastructure as code, in: ASE, 2022, pp. 1–12. https://doi.org/10.1145/3551349.3556945.

[11] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira, and A. Mendes, Polyglot code smell detection for infrastructure as code with GLITCH, in: ASE, 2023, pp. 2042–2045. https://doi.org/10.1109/ASE56229.2023.00162.

[12] D. Sokolowski, D. Spielmann, and G. Salvaneschi, Automated infrastructure as code program testing, IEEE Transactions on Software Engineering 50 (6) (2024) 1585–1599. https://doi.org/10.1109/TSE.2024.3393070.

[13] D. Spielmann, G. Zakhour, D. Arnold, M. Biagiola, R. Meier, and G. Salvaneschi, Metamorphic testing for infrastructure-as-code engines, Proceedings of the ACM on Programming Languages 10 (OOPSLA1) (2026), article 118. https://doi.org/10.1145/3798226.

[14] L. Madeyski, W. Orzeszyna, R. Torkar, and M. Józala, Overcoming the equivalent mutant problem: A systematic literature review and a comparative experiment of second order mutation, IEEE Transactions on Software Engineering 40 (1) (2014) 23–42. https://doi.org/10.1109/TSE.2013.44.

[15] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, Are mutants a valid substitute for real faults in software testing?, in: FSE, 2014, pp. 654–665. https://doi.org/10.1145/2635868.2635929.

[16] L. Inozemtseva and R. Holmes, Coverage is not strongly correlated with test suite effectiveness, in: ICSE, 2014, pp. 435–445. https://doi.org/10.1145/2568225.2568271.

[17] G. Petrović, M. Ivanković, G. Fraser, and R. Just, Does mutation testing improve testing practices?, in: ICSE, 2021, pp. 910–921. https://doi.org/10.1109/ICSE43902.2021.00087.

[18] G. Petrović, M. Ivanković, G. Fraser, and R. Just, Practical mutation testing at scale: A view from Google, IEEE Transactions on Software Engineering 48 (10) (2022) 3900–3912. https://doi.org/10.1109/TSE.2021.3107634.

[19] A. B. Sánchez, J. A. Parejo, S. Segura, A. Durán, and M. Papadakis, Mutation testing in practice: Insights from open-source software developers, IEEE Transactions on Software Engineering 50 (5) (2024) 1130–1143. https://doi.org/10.1109/TSE.2024.3377378.

[20] Aqua Security, tfsec, version 1.28.14, source repository and release. https://github.com/aquasecurity/tfsec/releases/tag/v1.28.14. Accessed 7 November 2026.

[21] HashiCorp, variable block reference: default and nullable, Terraform language documentation. https://developer.hashicorp.com/terraform/language/block/variable. Accessed 9 November 2026.

[22] Amazon Web Services, Configure the instance metadata options, Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-options.html. Accessed 9 November 2026.

[23] Microsoft, Azure Key Vault soft-delete overview: purge protection. https://learn.microsoft.com/en-us/azure/key-vault/general/soft-delete-overview. Accessed 9 November 2026.

[24] I. P. Ribeiro and V. de Oliveira Neves, Design of empirically-grounded mutation operators for Terraform IaC, Revista Eletrônica de Iniciação Científica em Computação 24 (2026). https://doi.org/10.5753/reic.2026.8471.